

CAMS Master Plan: Design to Final Project

Oct 6, 2026

1. Where the project stands

Planning ends with this document. Everything from the database to the final report is built and run in Claude Code, using `CLAUDE.md` for the rules and this plan for the design and the order of work.

Part	Status	Where it lives
Literature survey, 23 corrected references	Done	CAMS Report: Revised Sections 2 to 4
Limitations and problem statement	Done, rewritten	Same document
Hard rules, datasets, model and simulation approach	Decided	<code>CLAUDE.md</code>
System, data, model and simulation design	Designed in sections 2 to 6 below	This document
Build roadmap with prompts for Claude Code	Section 7 below	This document
Code, training, experiments, results, final report	Not started; done in Claude Code	Your repository

Before you stopped me, I started sketching a Flask app in my own scratch workspace. It was never run or tested, you cannot see it, and nothing in it is needed. Claude Code starts clean from the plan below.

What the finished project delivers

1. A working web app with patient, doctor and admin roles in which the same doctor, date and time can never be booked twice, enforced by the database.
2. A no-show prediction model trained on a public dataset and checked on a second one.
3. A simulation study comparing three booking policies and showing the trade-offs.
4. A risk flag in the app that shows staff which bookings are likely no-shows, without ever relaxing the slot rule.

5. A report in which every number comes from a script in the repository.

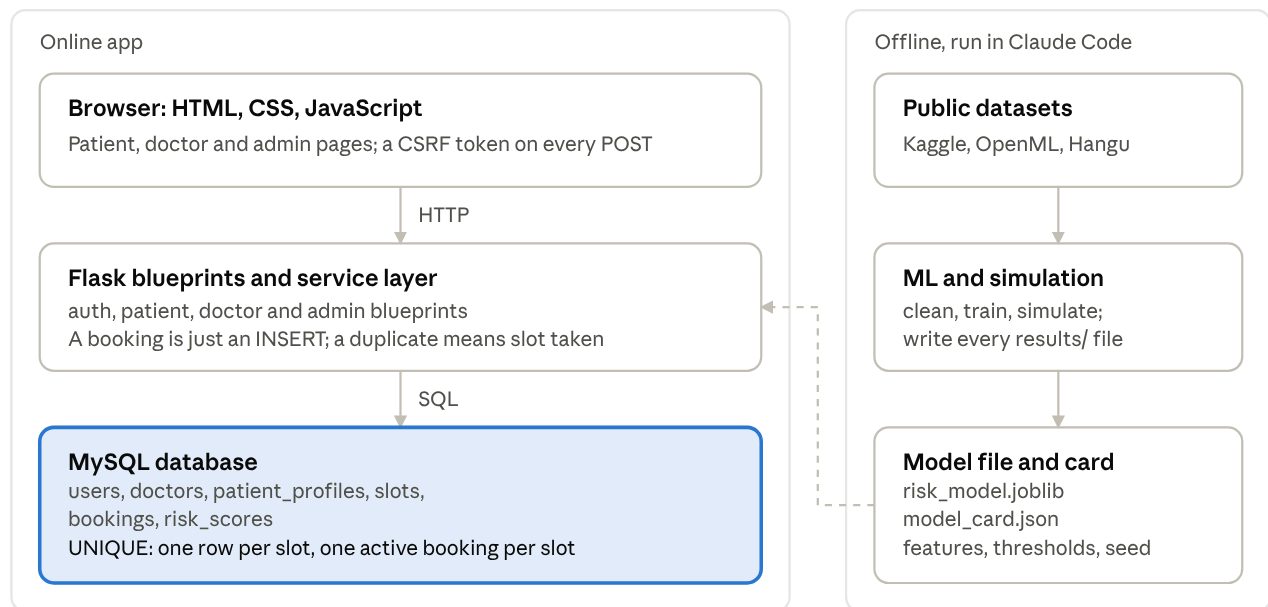
2. System design

CAMS is a three-tier Flask and MySQL application whose main guarantee, no double-booking, is enforced by the database rather than by a Python check.

Architecture

Browsers talk to Flask blueprints, which call a service layer that is the only code touching the database. The risk score from the model is read by the service layer and shown to staff as advice only.

The database, not the app, decides who gets a slot



Dashed line: the app reads the model score for staff pages only; booking never depends on it.

architecture · online app, offline work, one dashed link

Requests flow down the left column. The dashed line is the only link from the offline work into the app.

Roles

Role	Can	Cannot
Patient	Register, log in, browse doctors and free slots, book, cancel and reschedule own appointments	See other patients' bookings or any admin page
Doctor	See own schedule with patient names and the risk flag, cancel an appointment on own schedule, mark past appointments completed or no-show	Edit slots, see other doctors' schedules
Admin	Create doctor accounts, generate slots, see all bookings, cancel any booking	Book on a patient's behalf in version 1

Pages and routes

Route	Role	What it does
/register , /login , /logout	Anyone	Create a patient account, sign in, sign out
/patient/	Patient	List doctors with specialization
/patient/doctors/<id>?date=	Patient	Show free and taken slots for one day
/patient/book (POST)	Patient	Confirm a slot
/patient/bookings	Patient	Own upcoming and past appointments
/patient/bookings/<id>/cancel (POST)	Patient	Cancel own appointment
/patient/bookings/<id>/reschedule (POST)	Patient	Move to another slot in one transaction
/doctor/	Doctor	Seven-day schedule, previous and next week
/doctor/bookings/<id>/outcome (POST)	Doctor, Admin	Mark a past appointment completed or no-show

Route	Role	What it does
/admin/	Admin	Counts, doctor list, next 25 bookings
/admin/doctors/new	Admin	Create a doctor account
/admin/doctors/<id>/slots	Admin	Generate slots for a date range
cancel routes for doctor and admin	Doctor, Admin	Cancel an appointment they are allowed to manage

Data model

The submitted report shows three tables. The design needs four, because a slot must exist as its own row for the database to guard it.

Table	Key columns	Notes
users	id, name, email (unique), password_hash, role, created_at	One table for all three roles
patient_profiles	user_id (unique), date_of_birth, sex	Age and sex for patients only; the risk model needs both
doctors	id, user_id (unique), specialization, slot_minutes	Profile for a user whose role is doctor
slots	id, doctor_id, slot_date, slot_time	UNIQUE (doctor_id, slot_date, slot_time); created by the admin
bookings	id, slot_id, patient_id, status, reason, created_at, cancelled_at, cancelled_by, confirmed_slot_id	Status is confirmed, completed, no_show or cancelled; confirmed_slot_id is generated, see below
risk_scores (added in step 5)	booking_id, no_show_probability, model_version, scored_at	Advisory only, never read by booking logic

How double-booking is prevented

1. `slots` holds exactly one row per doctor, date and time, enforced by a UNIQUE constraint.
2. `bookings.confirmed_slot_id` is a generated column equal to `slot_id` for every status except cancelled, and NULL once it is cancelled. A UNIQUE constraint on it allows only one active booking per slot, so a completed or no-show appointment still holds its slot, while any number of cancelled bookings can sit beside it, because UNIQUE ignores NULLs in MySQL.
3. The app simply tries to insert a confirmed booking. If the database reports a duplicate, the app returns "slot just taken" (HTTP 409). It never checks first and then inserts, because that leaves a gap two requests can slip through.
4. Reschedule inserts the new booking and cancels the old one in one transaction. If the new slot is taken, everything rolls back and the old booking stays.
5. The concurrency test is the proof: many simultaneous requests for one slot must produce exactly one success. Claude Code must run it against your real MySQL version, since generated columns need MySQL 5.7 or later.

The submitted report's section 6.2 says the app checks for an existing BOOKED row before inserting. Replace that with the constraint design above.

Rules and security

- A slot whose start time has passed cannot be booked, and a started appointment cannot be cancelled.
- Passwords are hashed, never stored as text. Every page that changes data carries a CSRF token. Every route checks the role, and a patient asking for another patient's booking gets a 404.
- All database access goes through SQLAlchemy with bound parameters, so user input is never joined into SQL.
- The app never overbooks. Overbooking is studied only in the simulation.

3. Data plan

Three public datasets each have one job. You download them by hand, they stay out of git, and scripts clean them and write a log of every row removed.

Download steps

1. **Kaggle no-show data.** Search "Medical Appointment No Shows" on Kaggle (dataset `joniarroba/noshowappointments`), or run `kaggle datasets download -d joniarroba/noshowappointments` after creating a Kaggle API token. Put the CSV (`KaggleV2-May-2016.csv`) in `data/raw/kaggle/`.
2. **OpenML Medical-Appointment.** Dataset id 43617. Fetch it with `sklearn.datasets.fetch_openml(data_id=43617, as_frame=True)` and save a copy to `data/raw/openml/`. Its listed licence is GPL 2.
3. **Hangu service times.** Clone or download `github.com/fenghaolin/HanguData` (also on Zenodo, doi 10.5281/zenodo.7484205) into `data/raw/hangu/`. The repository holds several CSVs; open them and use the one with the `ServTime` column in seconds, then record which file in `data/README.md`. Licence CC BY-NC-SA 4.0, which allows coursework use with credit.

`data/README.md` must repeat these steps, the download date and each file's row count, so a teammate can rebuild everything from nothing.

Cleaning rules

Each rule becomes a function in `ml/src/data.py` that logs how many rows it changed to `results/cleaning_log.csv`.

Dataset	Check	Action
Kaggle	Negative age, implausible ages	Drop the rows and log the count
Kaggle	Appointment date earlier than scheduled date	Drop and log
Kaggle	Duplicate rows	Drop exact duplicates
Kaggle	Column typos (<code>Hipertension</code> , <code>Handcap</code>) and <code>PatientId</code> stored as a float	Rename; cast the ID to an integer
Kaggle	Target <code>No-show</code> is the text Yes or No	Convert to <code>no_show</code> = 1 for Yes
Kaggle	Lead time	Whole days from the scheduled date to the appointment date, ignoring the time of day

Dataset	Check	Action
OpenML	Target <code>show</code> is 1 for attended	<code>no_show = 1 - show</code>
OpenML	Cosine-encoded columns duplicate the discrete ones	Use the discrete month, weekday and hour columns only
OpenML	Negative lead days, duplicates, no patient ID	Drop and log; no patient-level features
Hangu	Service time of zero or below, extreme values	Inspect first, drop clear errors, keep the real long tail, convert seconds to minutes

The cleaning script, not you, reports the final row counts. The report quotes `results/cleaning_log.csv`.

Splits

- **Kaggle.** Sort by appointment date. The latest 20% of dates, cut at a date boundary and never mid-day, is the locked holdout. Tune only inside the older 80%, using GroupKFold on `PatientId` so one patient never sits on both sides.
- **OpenML.** Hold out the last month in the data. Use stratified cross-validation on the rest, since it has no patient ID.
- **History features.** A patient's earlier no-show count and rate use only appointments whose outcome was already known when the current booking was made, meaning their appointment date is before the current booking date. This is stricter than "before the current appointment" and is the version the live app can actually compute.
- **Seeds.** One fixed seed per script, written next to every result file.

Synthetic data for the app

`db/seed_synthetic.py` is the only source of app data, and it never reads the public datasets.

- 1 admin, 4 doctors across different specializations, about 40 patients with obviously fake names, and a shared dev password taken from an environment variable.
- Slots for the next 14 weekdays, 09:00 to 13:00 in each doctor's slot length.
- About a third of future slots booked at random; some cancelled.

- Past appointments with outcomes (completed or no-show) drawn from a simple per-patient propensity, so the risk flag has history to work with. A fixed seed makes the seed repeatable.
- Age and sex for each patient, drawn from stated distributions, never copied from a dataset.

4. Prediction design

The project trains two kinds of model: research models that use every feature a dataset offers, and one deployable model that uses only what the app can collect. The deployable model is the one the simulation and the staff flag use, so the study matches what a real clinic could run.

Features

Feature	Kaggle	OpenML	App can collect it	Used by
Lead time in days	Yes	Yes (latencia)	Yes	All models
Appointment weekday	Yes	Yes	Yes	All models
Age	Yes	Yes	Yes (from date of birth)	All models
Sex	Yes	Yes	Yes	All models
Earlier appointments and earlier no-shows	Yes (by PatientId)	No	Yes (from past bookings)	Kaggle models and deployable model
Reminder SMS received	Yes	No	No SMS in v1	Research only
Health flags, scholarship, neighbourhood	Yes	No	No	Research only

Feature	Kaggle	OpenML	App can collect it	Used by
Hour of day, booking channel, specialty	No	Yes	Hour yes, channel and specialty codes differ	OpenML research model only

- **Deployable model:** age, sex, lead time, weekday, earlier appointments and earlier no-show rate. Trained on Kaggle.
- **Transfer test:** age, sex, lead time and weekday only, since only those exist in both datasets.

Model ladder

1. Always predict show (the baseline that scores about 80% accuracy and is useless).
2. A rule baseline: predict no-show when the patient's earlier no-show rate is above a cut-off.
3. Logistic regression with balanced class weights.
4. Random forest.
5. Gradient boosting (scikit-learn's `HistGradientBoostingClassifier` ; XGBoost only if time remains).
6. Calibration of the best model with Platt scaling and isotonic regression, picking by Brier score on grouped cross-validation.

Every step must beat the one before on the metrics below, or the report says it did not.

Evaluation

- **Metrics:** AUC-ROC, average precision, sensitivity at fixed precision (30%, 40% and 50%), and Brier score. Accuracy is never the headline.
- **Uncertainty:** 1,000 bootstrap resamples of the holdout, resampling patients rather than rows, give a 95% interval on every metric.
- **Tuning:** small hyper-parameter grids, GroupKFold on `PatientId` , training data only. The holdout is scored once per final model.
- **Imbalance:** class weights and threshold choice first. Under-sampling, SMOTE and NearMiss appear only in an ablation, because they distort probabilities, and the simulation never uses a resampled model.

- **Reading the results:** expect an AUC roughly between 0.65 and 0.80, since Deina et al. reached 0.62 and 0.77 on their two datasets. Above 0.85, stop and hunt for leakage before reporting.
- **Reminder trap:** SMS may have been sent selectively, so compare models with and without it and cross-tabulate it against lead time before saying anything about its effect.

Experiments and the files each must write

ID	Question	Output in results/
E1	What do the datasets look like and what did cleaning remove?	<code>cleaning_log.csv</code> , <code>dataset_summary.csv</code> , EDA figures
E2	How do the two baselines score?	<code>e2_baselines.csv</code>
E3	Which model is best on Kaggle with all features?	<code>e3_kaggle_models.csv</code> , ROC and precision-recall figures
E4	Which feature groups add the most?	<code>e4_ablation.csv</code>
E5	Do the same findings hold on OpenML?	<code>e5_openml_models.csv</code>
E6	How much AUC is lost when training on one dataset and testing on the other?	<code>e6_transfer.csv</code>
E7	Do resampling methods beat class weights and thresholds?	<code>e7_imbalance.csv</code>
E8	Is the best model calibrated, and does calibration help?	<code>e8_calibration.csv</code> , reliability figure
E9	How does the deployable model score?	<code>e9_deployable.csv</code> , model file with a model card
E10	What precision and sensitivity does each risk threshold give?	<code>e10_thresholds.csv</code>

Every result file is written next to a `manifest.json` entry holding the seed, the data file's hash, the date and the git commit.

Leakage and correctness tests

- The history feature for a booking never uses an outcome whose appointment date is on or after that booking's date.
- No patient appears in both a training fold and its validation fold, and the holdout dates all come after the training dates.
- Both loaders give no-show = 1 for the same hand-made rows.
- Preprocessing and encoders are fitted inside each training fold only.
- A model trained on shuffled labels scores an AUC near 0.5.

5. Simulation design

A SimPy discrete-event model of one doctor's half-day session compares three booking policies and reports the trade-offs as a table, never as one weighted score.

What is simulated

- One doctor and one session of T minutes, split into slots of L minutes. $N = T / L$ slots are booked normally.
- Every patient arrives exactly at their slot time. Patients are seen first come, first served by slot time.
- Each patient has a true no-show probability p , taken from the calibrated deployable model. Whether they attend is drawn from p . A no-show uses no doctor time.
- A consultation takes a time drawn from the fitted service-time distribution. Service starts when the patient has arrived and the doctor is free.
- A double-booked slot has two patients arriving at the same time; the second waits behind the first.
- Defaults for T and L come from the seeded app slots. The default L is Hangu's median service time rounded up to the next 5 minutes, calculated by the script, not typed.

Inputs

Input	Source	How
Consultation time	Hangu <code>ServTime</code> , in minutes	Fit lognormal and gamma by maximum likelihood, choose by AIC and QQ plots, and state the choice
Patient risk <code>p</code>	Kaggle holdout rows scored by the deployable model	Draw patient profiles at random with replacement for each session
Session shape	Parameters	<code>T</code> , <code>L</code> , <code>N</code> are explicit and listed in the results manifest
Randomness	Seeded generator	Replication <code>r</code> uses <code>base_seed + r</code> ; every policy sees the same draws (common random numbers)

Policies

Policy	Rule
P0	No overbooking: one patient per slot
P1	Uniform: every k-th slot gets a second patient, for k = 2, 3, 4 and 6
P2	Risk threshold: a slot gets a second patient when the first patient's predicted risk is at least a threshold. The threshold grid uses fixed values 0.3 and 0.5 plus the 50th, 70th, 80th, 90th and 95th percentiles of predicted risk, so the rule actually triggers when base rates are near 20%
Stretch	A simple reinforcement-learning policy, only if time remains, following the Amalina & An preprint

For each replication the script draws patients, attendance and service times once, for a primary patient and an extra patient per slot, then lets each policy pick which extras to use. That makes policies directly comparable.

Metrics, per policy

Patients served, mean waiting time of attending patients, doctor overtime past T , doctor idle time before T , and the share of sessions in which both patients of a double-booked slot attended. Each is reported with a 95% interval from at least 1,000 replications; differences from P0 use the paired replications, which makes the intervals tight.

Experiments

ID	Question	Output in results/
S1	Does the simulator behave sensibly?	Sanity tests, see below
S2	What does each policy trade away at default settings?	s2_policy_tradeoffs.csv , frontier figures
S3	How much does a biased risk estimate hurt P2? Shift predictions by -0.10, -0.05, 0, +0.05, +0.10	s3_probability_error.csv
S4	What if the base no-show rate is lower or higher? Scale it by 0.5, 1 and 1.5	s4_base_rate.csv
S5	What if consultations are more or less variable? Scale variability by 0.5, 1 and 1.5 at the same mean	s5_service_variability.csv
S6	Is the model worth having? Compare P2 against P1 and against random scores at the same number of double-booked slots, and against an oracle that knows who will not attend	s6_value_of_prediction.csv

S6 is the headline analysis. A policy that overbooks more will always serve more patients, so only a comparison at equal overbooking rates shows whether prediction adds anything.

Sanity tests

- If everyone attends and service time equals L , nobody waits and there is no overtime.
- If nobody attends, zero patients are served and idle time equals T .
- A hand-worked three-patient session gives exactly the simulator's numbers.

- The same seed gives identical results, and PO results do not change when other policies are added.

Assumptions to state in the report

Patients arrive on time, there are no walk-ins, there is one doctor, service time does not depend on the patient, and no-show behaviour comes from a Brazilian public-clinic dataset while consultation times come from a Chinese private traditional-medicine clinic. The simulation shows a method and its trade-offs, not clinical outcomes.

6. Joining it together: risk flag, tests and acceptance

The deployable model is exported once, loaded by the Flask app, and shown to doctors and admins as an advisory flag. Booking logic never reads it, so a model failure can never block or change a booking.

Risk flag design

1. **Model file.** The training script writes `ml/artifacts/risk_model.joblib` and a `model_card.json` holding the feature list, calibration method, training data hash, seed, version and the risk thresholds from experiment E10.
2. **One feature function.** `ml/src/features.py` builds the six deployable features. The app imports that same function, so training and serving cannot drift apart. A test feeds one booking through both paths and expects identical numbers.
3. **When it is scored.** Right after a booking is committed, and again for any booking shown without a score. Result goes into `risk_scores`. If the model file is missing or scoring fails, the app logs it and shows no flag; the booking is unaffected.
4. **What staff see.** A Low, Medium or High badge on the doctor schedule and the admin booking list. High and Medium come from the thresholds that gave 40% and 30% precision on the holdout, taken from the model card, not typed in.
5. **What patients see.** Nothing. The wording on staff pages says the flag is advisory and is never a reason to refuse or move a booking.
6. **Stretch, only if time remains.** A standby list for patients who want an earlier slot if one is cancelled.

The model uses age and sex as inputs. The report should say so under limitations, explain that the flag only informs staff, and note that no patient is ever denied a slot because of it.

Test plan

Area	What the tests prove
Booking	Two simultaneous requests for one slot give exactly one success; six requests across 25 slots give one winner per slot; a direct SQL insert of a second active booking fails; a cancelled slot can be rebooked; reschedule is all-or-nothing
Accounts and roles	Passwords are hashed; each role is blocked from the others' pages; every POST needs a CSRF token; a patient cannot see or cancel someone else's booking
Slots	Generation can be run twice without duplicates, weekends are skipped when asked, past slots cannot be booked
Data and models	The leakage tests in section 4, plus correct no-show polarity for both datasets
Simulation	The sanity tests in section 5
Risk flag	Training and serving features match; a missing model does not break booking; the flag never changes which slots are free
End to end	Seed the database, log in as each role, book, and see the flag

The booking tests run twice, once on SQLite and once on a real MySQL database, because the generated-column constraint must be proven on the engine the project actually uses.

Acceptance checks

- ☐ The concurrency test passes on MySQL, not only on SQLite
- ☐ One command runs the whole test suite and it passes
- ☐ One command rebuilds every file in `results/` from the raw data with fixed seeds
- ☐ One command rebuilds the simulation results
- ☐ Every number in the report appears in a results file, checked against a numbers audit table

- ☐ No raw dataset is in git and no real patient data is in the app
- ☐ A teammate can set the project up from the README alone

7. Build roadmap for Claude Code

The build is nine milestones, each one Claude Code session, and each ends with something you can run. Export this document as Markdown into the repository as `docs/PLAN.md` first, so Claude Code can read it.

How to run every session

- Start by telling Claude Code to read `CLAUDE.md` and the section of `docs/PLAN.md` for that milestone.
- One milestone per session, tests before code where the plan says so, and a git commit when the milestone's checks pass.
- Claude Code never types a number into the report or a results file. If a result is missing, it runs the script.
- If a hard rule seems to be in the way, Claude Code stops and asks you instead of bending it.

Milestones and prompts

Milestone	Week	Prompt to give Claude Code	Done when
M0 Setup	1	"Read CLAUDE.md and docs/PLAN.md fully. Create the repository layout from CLAUDE.md, a virtual environment, requirements.txt, a .gitignore that excludes data/raw, and a README with MySQL setup steps. Write no features yet; show me the tree and wait."	The tree exists and the app can connect to your MySQL

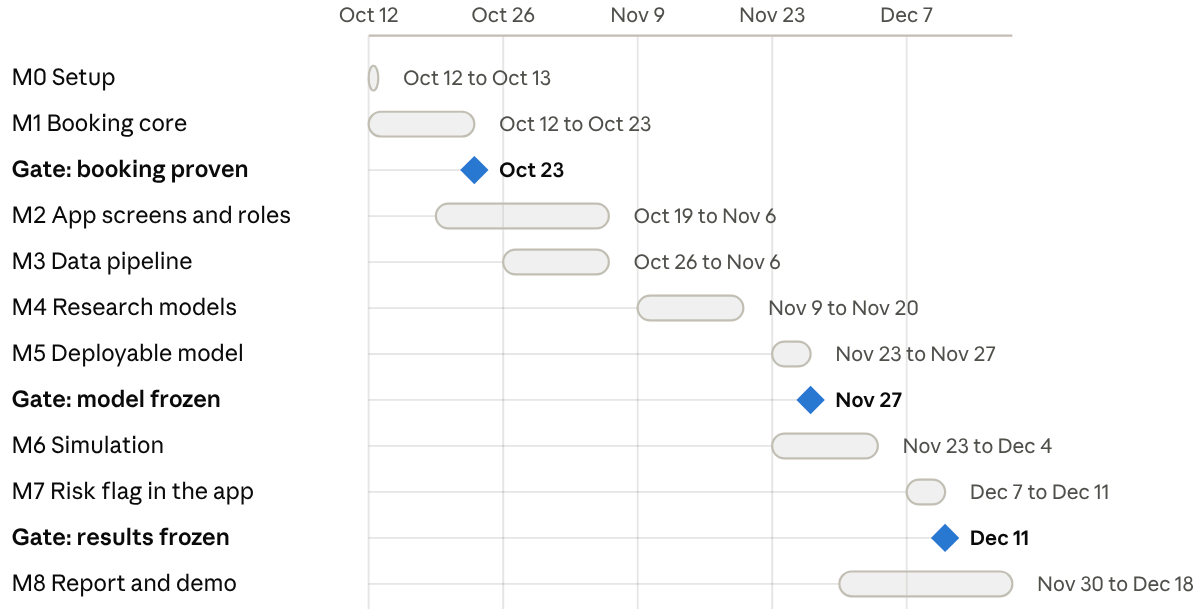
Milestone	Week	Prompt to give Claude Code	Done when
M1 Database and booking core	1 to 2	"Build the database and booking core from PLAN section 2. Write the concurrency and constraint tests first and show them failing, then implement the schema, models and services until they pass on SQLite and on my MySQL."	Concurrency test passes on MySQL
M2 App screens and roles	2 to 4	"Build the Flask app from PLAN section 2: auth, patient, doctor and admin blueprints, CSRF protection, templates and db/seed_synthetic.py. Add the role, slot and end-to-end tests from PLAN section 6."	You can log in as each role and book, cancel and reschedule
M3 Data pipeline	3 to 4	"Build the data pipeline from PLAN section 3: loaders, one cleaning function per rule with a logged row count, the split code, and experiment E1. I will place the raw files in data/raw myself."	<code>cleaning_log.csv</code> and <code>data/README.md</code> exist
M4 Research models	5 to 6	"Run experiments E2 to E8 from PLAN section 4. Write the leakage tests first. Every result goes to results/ with a manifest entry. Stop and tell me if any validation AUC is above 0.85."	E2 to E8 files exist and leakage tests pass
M5 Deployable model	7	"Build the deployable model and experiments E9 and E10, export the model file and model card, and put the shared feature function in ml/src/features.py."	Model file, card and threshold table exist

Milestone	Week	Prompt to give Claude Code	Done when
M6 Simulation	7 to 8	"Build the SimPy simulator and experiments S1 to S6 from PLAN section 5. Write the sanity tests first. Use common random numbers and at least 1,000 replications."	S1 to S6 files exist and sanity tests pass
M7 Risk flag in the app	9	"Integrate the risk flag from PLAN section 6: app/risk.py, the risk_scores table, staff badges, and the tests for training-serving agreement and for a missing model."	Flag shows on staff pages and booking is unaffected without the model
M8 Report and demo	8 to 10	"Produce every table and figure the report plan lists from results/, build the numbers audit table, and draft the Results and Limitations chapters from the result files only."	Every report number traces to a results file

The weeks assume you start on Monday 12 October 2026 and have ten weeks, matching your submitted report's ten-week plan. Your real deadline decides whether that holds; the timeline below is a proposal to adjust.

Models freeze in week 7; results freeze in week 9

Proposed plan from Monday 12 October 2026, ten weeks; adjust to your deadlines



build timeline · 9 milestones, 3 gates

Diamonds are gates. Booking is proven when the concurrency test passes on MySQL, the model is frozen when the E9 and E10 files exist, and results are frozen when no experiment is rerun after that date.

If time runs short

Cut in this order: the reinforcement-learning policy, the standby list, XGBoost, the resampling ablation (E7). Never cut the booking constraint and its tests, experiments E3, E8 and E9, simulation S2 and S6, or the numbers audit.

8. Report plan

The final report keeps your submitted structure for sections 1 to 5 and adds three chapters for implementation, results and limitations. Three of the existing sections are already rewritten.

Chapter changes

Section in the submitted report	What changes
1 Introduction	Add no-shows and overbooking to the motivation, add prediction and simulation to the project vision, and update the last paragraph that lists the sections
2 Literature Survey	Already replaced: 23 references in five themes
3 Limitations of the Existing Works	Already replaced
4 Problem Statement	Already replaced, with three research questions
5 Objectives	Keep the nine; reword objective 5 from "server-side validation" to a database constraint; add four: train and evaluate a no-show model on public data, check it on a second dataset, simulate overbooking policies and report trade-offs, add an advisory risk flag
6 Proposed Design	Replace the three-table schema with the four-table one; replace "check for a BOOKED row before INSERT" with the constraint design; add the data, model and simulation design from this plan; replace the ten-week table with the real timeline
7 Implementation and Testing (new)	Stack, screenshots of each role's pages, the test summary and the concurrency test evidence
8 Results (new)	Every table and figure in the list below, each quoted from a results file
9 Limitations and Future Work (new)	The limitations below, then SMS reminders, a real-clinic pilot, a multi-doctor simulation and reinforcement learning
10 Conclusion	Rewrite "Expected Outcome" in the past tense using actual results
11 References	The 23 corrected entries

Tables and figures to produce

Item	Comes from
Dataset summary and cleaning log	E1
Architecture diagram, ER diagram, booking flow, role screenshots	Drawn by the team; screenshots taken from the running app
Test summary (tests run, passed)	<code>results/test_summary.txt</code> , written by a script from the test run
Model comparison with confidence intervals; ROC and precision-recall curves	E3
Feature-group ablation	E4
OpenML results and the transfer table with AUC drop	E5, E6
Imbalance handling comparison	E7
Calibration curves and Brier scores	E8
Deployable model scores and threshold table	E9, E10
Policy trade-off table and frontier figures	S2
Sensitivity tables	S3, S4, S5
Value of prediction at equal overbooking	S6

Numbers audit

Keep `docs/numbers_audit.csv` , with one row per number quoted in the report: the number, the results file and the column it comes from. A script re-reads each file and fails if any quoted value no longer matches. Run it before every submission.

Limitations to state plainly

- No real clinic data: the model learned from a Brazilian public-clinic dataset, consultation times come from a Chinese private clinic, and the app runs on synthetic records.

- The datasets are short: the Kaggle data covers about six weeks and OpenML about four months, so seasonal patterns are not captured.
- The simulation covers one doctor and punctual patients, so its results show a method and trade-offs, not clinical outcomes.
- The deployable model uses only age, sex, lead time, weekday and history, and it has not been tested on live clinic traffic.
- Using age and sex raises fairness questions; the flag only informs staff.
- Amalina and An (2026) is an arXiv preprint and has not been peer reviewed.

9. Risks, open items and viva preparation

The biggest technical risk is the booking constraint behaving differently on your MySQL version, and the biggest honesty risk is a model that looks too good.

Risks and fallbacks

Risk	Fallback
Generated-column constraint is not supported or behaves differently on your MySQL	Add a <code>slot_holds</code> table whose primary key is <code>slot_id</code> , inserted and deleted in the same transaction as the booking; keep the same concurrency test
Kaggle needs a login, so Claude Code cannot download it	You download it by hand; the README records the steps
The OpenML or Hangu files differ from their descriptions	Claude Code inspects the real columns first and reports mismatches before writing code
Validation AUC above 0.85	Treat it as leakage: stop, find the cause, fix it, rerun
AUC near 0.65	Report it honestly; S6 then shows how little prediction adds, which is a valid finding
Reminder SMS looks harmful or helpful	Compare with and without it and describe the link as an association, not a cause
P2 shows no gain over P1 at equal overbooking	Report it; a negative result with a fair test is still a result

Risk	Fallback
Time runs short	Use the cut list in section 7

Decisions added since CLAUDE .md

The updated CLAUDE .md delivered with this plan already includes these.

- Booking statuses are confirmed, completed, no_show and cancelled. Only cancelled frees the slot.
- A patient_profiles table holds date of birth and sex.
- The deployable model uses six features, and the simulation uses that model.
- History features use only outcomes known when the booking was made.
- Simulation S6 compares P2 with P1 at equal overbooking.
- Every result has a manifest entry, and the report has a numbers audit.
- A fallback exists if the generated-column constraint fails.

Open items

- ☐ Confirm your real submission and demo dates; this plan assumes ten weeks from 12 October 2026
- ☐ Confirm with Dr. Roy that the ML and simulation work and the added chapters are in scope
- ☐ Check your MySQL version: 5.7 or later, or MariaDB 10.2 or later
- ☐ Download the three datasets and note the date
- ☐ Citations to confirm: Fan et al. pages 469 to 490, and the full author lists for Deina et al. and Zhou et al.
- ☐ Read the papers behind the entries you rely on most: Gupta and Denton, LaGanga and Lawrence, Samorani and LaGanga, Carreras-García et al., Zhou et al.
- ☐ Check your college library access for Elsevier, Wiley and INFORMS papers

Viva questions to prepare

Question	Answer cue
Why enforce no-double-booking in the database?	An app check leaves a gap between check and insert that two requests can both pass; the constraint is atomic, and the concurrency test shows it
Why not report accuracy?	Always predicting show scores about 80%
Why not use SMOTE in the final model?	It distorts probabilities, and the simulation needs calibrated ones
What does the model add over a simple rule?	Compare E2 with E3 and E9, quoting the result files
Why does the app not overbook?	Overbooking is a clinic policy decision with real costs; the app only advises, and the simulation shows the trade-offs
Why combine data from Brazil and China?	No single public dataset has both; it is stated as an assumption and tested by sensitivity runs
Is using age and sex fair?	It is advisory only, nobody is refused a slot, and E4 shows how much demographics contribute
What is new here?	An open, reproducible pipeline joining a booking app, a prediction model and a policy simulation, not a new algorithm
How do you know the numbers are right?	Fixed seeds, scripts that write every result, and the numbers audit